📖 Nivel Intermedio UI: Definiciones y teoría

1. **Expanded**

- **Definición**: Ocupa todo el espacio disponible dentro de un `Row` o `Column`.
- **Teoría**: Se usa para que un widget se expanda proporcionalmente en relación con otros elementos.

---

2. **Flexible**

- **Definición**: Similar a `Expanded`, pero permite que el hijo se ajuste sin forzar el tamaño exacto.
- **Teoría**: Da más control sobre cómo los widgets se adaptan dentro de un layout.

---

3. **SingleChildScrollView**

- **Definición**: Hace que un único hijo sea desplazable.
- **Teoría**: Útil cuando el contenido puede desbordar la pantalla en dispositivos pequeños.

---

4. **ListView**

- **Definición**: Lista desplazable de múltiples elementos.
- **Teoría**: Fundamental para mostrar colecciones de datos (productos, mensajes, etc.).

---

5. **MediaQuery**

- **Definición**: Permite acceder a información del dispositivo (ancho, alto, orientación).
- **Teoría**: Es la base para construir interfaces responsivas.

---

6. **LayoutBuilder**

- **Definición**: Construye widgets en función de las restricciones de su contenedor padre.
- **Teoría**: Permite definir distintos layouts según el espacio disponible.

💻 Ejemplo completo de aplicación (nivel intermedio)

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
```

```
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Curso Flutter - Nivel Intermedio',
      theme: ThemeData(primarySwatch: Colors.teal),
      home: const ResponsiveScreen(),
    );
  }
}

class ResponsiveScreen extends StatelessWidget {
  const ResponsiveScreen({super.key});

  @override
  Widget build(BuildContext context) {
    // Obtenemos dimensiones de la pantalla
    final size = MediaQuery.of(context).size;
    final orientation = MediaQuery.of(context).orientation;

    return Scaffold(
      appBar: AppBar(
        title: const Text('Pantalla Responsiva'),
        centerTitle: true,
      ),
      body: LayoutBuilder(
        builder: (context, constraints) {
          // Cambiamos el layout según el ancho disponible
          if (constraints.maxWidth < 600) {
            // Pantallas pequeñas (teléfonos)
            return SingleChildScrollView(
              child: Column(
                children: [
                  Container(
                    color: Colors.teal[100],
                    height: 150,
                    child: const Center(child: Text('Bloque 1')),
                  ),
                  Container(
                    color: Colors.teal[200],
                    height: 150,
                    child: const Center(child: Text('Bloque 2')),
                  ),
                  Container(
                    color: Colors.teal[300],
                    height: 150,
                    child: const Center(child: Text('Bloque 3')),
                  ),
                ],
              ),
            );
          } else {
            // Pantallas grandes (tablets)
            return Row(
              children: [
                Expanded(
```

```
                    child: Container(
                      color: Colors.teal[100],
                      height: 300,
                      child: const Center(child: Text('Bloque 1')),
                    ),
                  ),
                  Expanded(
                    child: Container(
                      color: Colors.teal[200],
                      height: 300,
                      child: const Center(child: Text('Bloque 2')),
                    ),
                  ),
                  Flexible(
                    flex: 2,
                    child: Container(
                      color: Colors.teal[300],
                      height: 300,
                      child: const Center(child: Text('Bloque 3 (Flexible)')),
                    ),
                  ),
                ],
              );
            }
          },
        ),
      );
    }
  }
```

🎯 Resumen:

1. **Uso de Expanded y Flexible** para distribuir espacio dinámicamente.
2. **Scroll adaptativo** con `SingleChildScrollView` y `ListView`.
3. **Detección de tamaño y orientación** con `MediaQuery`.
4. **Construcción condicional de layouts** con `LayoutBuilder`.
5. **Primeros pasos hacia interfaces realmente responsivas** (ejemplo: cambiar de `Column` a `Row` según el ancho).